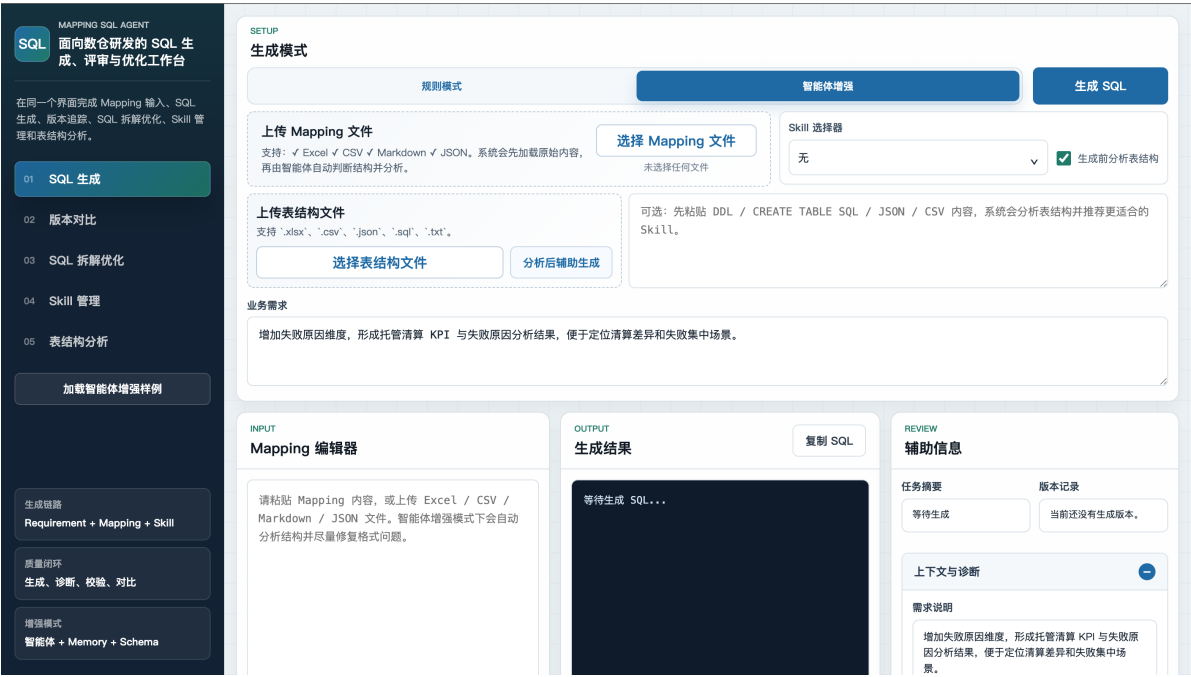


Mapping SQL Agent

项目演示材料



面向评委审阅的系统功能说明与页面演示材料

内容定位：总体功能介绍、页面组件说明、样例加载说明、系统亮点归纳

1. 材料定位与阅读说明

本材料目标是以相对完整的方式说明 Mapping SQL Agent 的系统定位、页面结构、功能模块、样例数据和业务价值。本材料先建立系统整体认知，再逐步说明各模块页面和关键组件，最后归纳系统亮点与可扩展方向。

Mapping SQL Agent 是一个面向数仓研发场景的本地智能体工作台。系统围绕数据开发中的 Mapping 文档、业务需求、SQL 生成、版本变更、SQL Review 和表结构理解展开，将多个原本分散在文档、脚本和人工评审中的环节集中到一个可交互页面中。当前版本不直接连接真实数据库，也不执行建表或跑数，而是以文档型输入为核心，完成解析、生成、诊断、对比和优化。

2. 系统总体功能介绍

2.1 页面整体结构

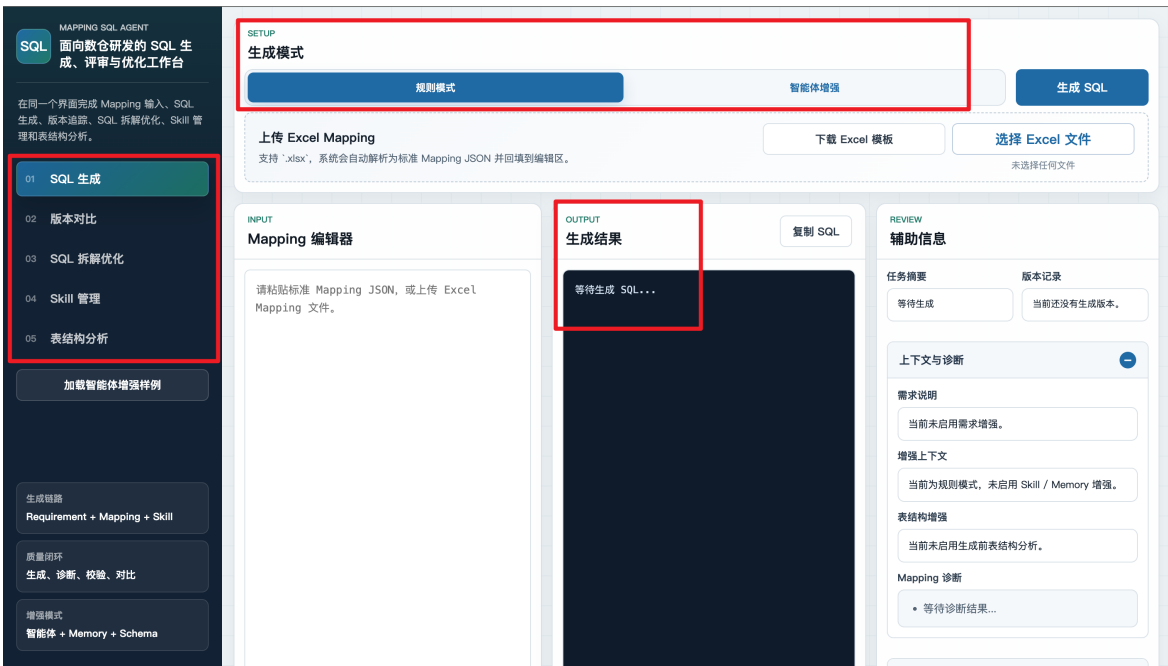


图 1: Mapping SQL Agent 工作台整体页面

系统页面采用左侧导航和右侧工作区的结构。左侧包含 SQL 生成、版本对比、SQL 拆解优化、Skill 管理和表结构分析五个核心工作区，下方通过生成链路、质量闭环和增强模式说明系统定位。右侧区域根据当前功能动态切换，通常由输入配置区、样例加载区、结果展示区和辅助诊断区组成。

这种页面组织方式体现了项目的工作台属性。用户不是在多个割裂工具之间切换，而是在同一个界面中完成从需求输入、文件上传、规则生成、智能体增强、版本追踪、SQL Review、Skill 沉淀到表结构分析的连续流程。对于评审者而言，整体页面能够直观看到系统覆盖的数据研发环节，而不仅是单点 SQL 生成能力。

2.2 五个核心模块

系统的五个核心模块按照数据研发 workflow 组织。SQL 生成模块负责把需求和 Mapping 转换为 SQL 初稿；版本对比模块负责追踪需求变化带来的 Mapping 和 SQL 差异；SQL 拆解优化模块负责阅读、分析和优化已有 SQL；Skill 管理模块负责把业务经验沉淀为可复用上下文；表结构分析模块负责识别字段角色和表用途。五个模块共同覆盖“生成、对比、优化、沉淀、理解”五类能力。

模块	功能定位	核心输入	主要输出
SQL 生成	从 Mapping、需求、Skill 和表结构上下文生成 SQL 初稿。	Mapping JSON、Excel、CSV、Markdown、TXT、自然语言需求、表结构。	SQL 代码、任务摘要、版本记录、诊断信息、规则校验结果。
版本对比	比较历史版本和当前版本的 Mapping 与 SQL 差异。	历史版本、当前 Mapping、当前需求、生成模式。	SQL 差异、Mapping 影响、历史版本说明、当前版本结果。
SQL 拆解优化	对已有 SQL 进行结构理解、作用分析和优化建议生成。	SQL 文件或文本、Skill、可选表结构。	SQL 作用分析、结构拆解、优化建议、优化后 SQL。
表结构分析	把 DDL、Excel、CSV、JSON 或文本表结构转化为可理解的元数据信息。	CREATE TABLE DDL、表结构文件、字段说明文本。	表用途分析、关键字段识别、结构整理、可复用建议。

2.3 样例数据设计

系统内置样例围绕真实数仓业务进行设计，当前版本重点采用托管清算业务口径来组织演示数据。SQL 生成样例使用托管产品清算汇总和托管清算对账场景，包含托管清算明细事实表、交易明细事实表、产品维表、业务日期分区、清算状态过滤、清算金额、交易金额、差异金额、清算笔数和失败笔数等内容。版本对比样例使用 dws_custody_clearing_compare_demo，模拟从按产品汇总清算金额，到增加清算状态和交易金额核对，再到补充失败原因、失败笔数和清算 KPI 的需求演进。SQL 拆解优化样例使用托管清算对账 SQL，并结合托管清算对账 Skill 与表结构上下文。表结构分析样例使用托管清算明细表 DDL，覆盖清算流水、交易流水、产品、账户、清算状态、金额差异、失败原因和分区日期等字段。

样例设计价值

这些样例不是单纯用于填充页面，而是用于展示系统如何处理事实表、维表、Join、过滤条件、聚合指标、分区字段、业务维度和字段注释等真实数据开发元素。通过样例，评委可以看到系统对数仓研发流

程的覆盖程度和业务贴合度。

3. SQL 生成模块

3.1 模块功能说明

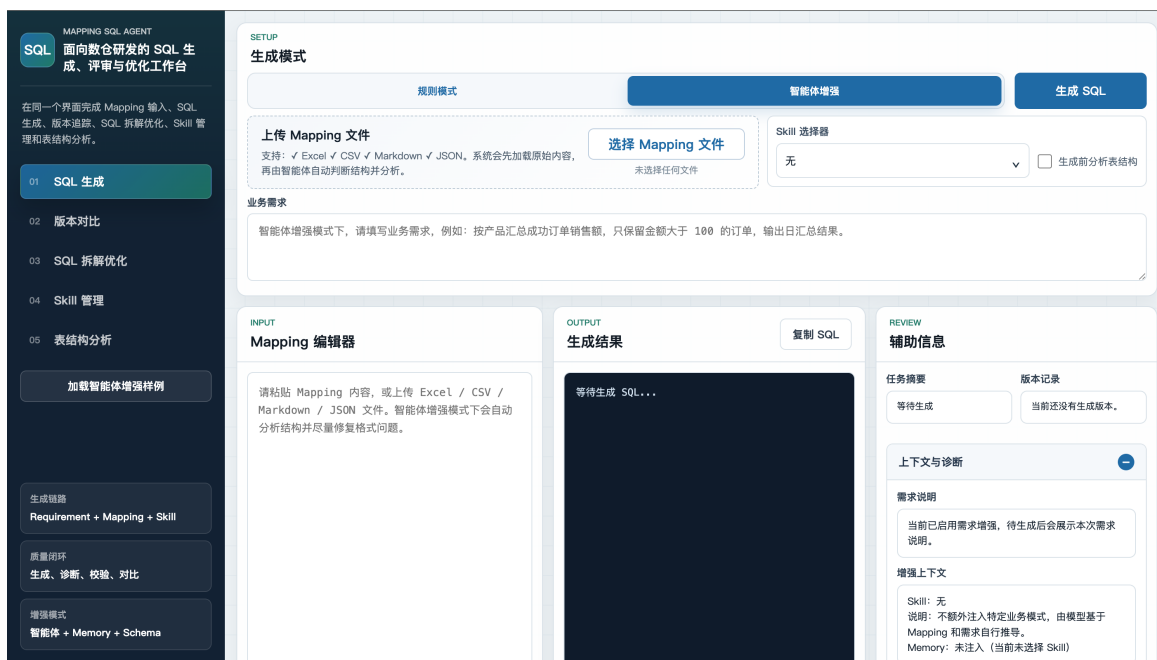


图 2：SQL 生成工作区整体页面

SQL 生成模块被设计为系统的主流程入口，原因在于数仓研发的起点通常不是直接编写 SQL，而是先接收业务需求、Mapping 文档、表结构说明和历史口径约束。开发人员需要把这些材料转化为目标表、来源表、字段映射、Join 条件、过滤条件、聚合口径和分区写入逻辑，最终形成可执行 SQL。这个过程重复性强、细节多，并且容易因为字段理解偏差或口径遗漏产生问题。因此，SQL 生成模块的核心设计目标不是简单输出一段 SQL，而是把“需求理解、结构化 Mapping、业务口径、表结构上下文、结果诊断”组织成一条可解释的生成链路。

从逻辑上看，该模块将 SQL 生成拆成了两层能力。第一层是模板生成，也就是基于本地规则引擎的确定性生成。系统读取标准 Mapping 中的 target_table、sources、joins、filters、target_columns 等字段，生成稳定的 WITH、INSERT OVERWRITE、SELECT、JOIN、WHERE 和 GROUP BY 结构。这一层对应真实业务中格式规范、字段映射清晰的常规开发任务，优势是结果稳定、来源可追溯、便于评审。第二层是智能体增强生成，也就是在规则草稿基础上加入用户需求、Skill、Memory 和表结构分析结果，让模型在明确上下文约束

下补充业务语义和优化表达。这一层对应真实业务中需求描述不完整、Mapping 格式不统一、字段含义需要结合表结构理解的复杂场景。

页面布局也围绕这一逻辑展开。顶部的生成模式区域用于区分规则模式和智能体增强模式，体现系统同时支持稳定生成和智能增强。Mapping 上传与编辑区域用于承接真实工作中常见的字段映射材料，既支持标准 Mapping，也支持 Excel、CSV、Markdown、JSON、TXT 等更贴近业务交付形态的文件。业务需求输入区用于补充自然语言口径，使系统能够理解“按什么维度统计、统计什么指标、核验哪些金额、过滤哪些清算状态、输出到什么粒度”等信息。Skill 选择器用于把托管清算对账、估值核算、资金划拨、余额对账等业务经验显式注入生成过程，生成前表结构分析则用于补足字段角色理解。

这种设计符合实际数据研发流程。真实项目中，开发人员往往需要先看需求，再看 Mapping，再查表结构，最后结合团队口径写 SQL；生成后还要检查字段是否覆盖、分区是否遗漏、Join 是否合理、指标口径是否符合业务要求。SQL 生成模块把这些人工步骤拆解为页面组件和后端处理链路，使评审者能够看到输入材料如何被组织、规则草稿如何形成、智能体模型如何在 Skill 和表结构约束下增强 SQL，以及生成结果如何通过辅助信息进行诊断和复核。

3.2 样例加载内容

模板生成样例会加载 dws_custody_product_clearing_day_demo，来源表包括 dwd_custody_clearing_detail_di 和 dim_custody_product_df，过滤条件包括业务日期分区和清算状态，输出字段包括产品编号、产品名称、清算金额、清算笔数和失败笔数。智能体增强样例会加载 dws_custody_clearing_reconcile_day_demo，在清算明细基础上加入交易明细表 dwd_custody_trade_detail_di 和产品维表 dim_custody_product_df，需求说明要求按产品和清算状态统计交易金额、清算金额、差异金额、清算笔数和失败笔数，并选择“托管清算对账”Skill，同时启用生成前表结构分析。

3.3 关键组件说明

该模块的关键组件包括生成模式切换、Mapping 上传与编辑、业务需求输入、Skill 选择器、生成前表结构分析、SQL 输出区和辅助信息区。生成模式切换用于区分规则模式和智能体增强模式；Mapping 编辑器承接标准或非标准输入；Skill 选择器和表结构分析用于提供业务与元数据上下文；辅助信息区用于展示需求说明、增强上下文、Mapping 诊断、规则校验和字段检查。

SETUP 生成模式	生成模式与操作按钮 该区域用于选择规则模式或 智能体增强模式，并提供加载样例、生成 SQL 等操作入口。规则模式强调稳定
----------------------------------	--

规则模式 DeepSeek 增强 图 3：生成模式与操作按钮	可解释，增强模式强调业务上下文理解。
INPUT Mapping 编辑器 请粘贴 Mapping 内容，或上传 Excel / CSV / Markdown / JSON 文件。DeepSeek 模式下会自动分析结构并尽量修复格式问题。 图 4：Mapping 编辑器	Mapping 编辑器 该区域展示目标表、来源表、Join、过滤条件和目标字段映射，是 SQL 生成的结构化依据。标准 Mapping 可直接被规则引擎解析。
OUTPUT 生成结果 复制 SQL 等待生成 SQL... 图 5：结果区	结果区 该区域展示生成 SQL 并可直接复制。
REVIEW 辅助信息 任务摘要 版本记录 上下文与诊断 需求说明 增强上下文 表结构增强 Mapping 诊断 规则与校验 修复后 Mapping 规则草稿 SQL 图 6：辅助信息区	辅助信息区 该区域展示任务摘要、版本记录、Mapping 诊断和校验信息，用于帮助评审人员判断生成结果是否完整可靠。
业务需求 DeepSeek 增强模式下，请填写业务需求，例如：按产品汇总成功订单销售额，只保留金额大于 100 的订单，输出日汇总结果。 图 7：需求输入区	需求输入区 该组件允许用户输入自然语言业务需求，使系统能够理解统计口径、过滤范围、输出指标和业务目标，而不是只依赖字段映射。
Skill 选择器 无 生成前分析表结构 图 8：Skill 与增强配置区	Skill 与增强配置区 该组件用于选择产品维度汇总、同比、环比、用户分层和 KPI 统计等业务 Skill，使生成结果更贴近真实业务场景。
Skill 选择器 产品维度汇总 生成前分析表结构 上传表结构文件 选择表结构文件 分析后辅助优化	生成前表结构分析区 该组件用于上传或粘贴表结构材料，系统先分析表用途和关键字段，再将分析结果注入 SQL 生成上下文。

图 9：生成前表结构分析区

3.4 模块亮点

该模块的亮点在于同时保留规则生成和模型增强两条路径。规则生成保证 SQL 骨架稳定、字段来源可追溯；智能体增强则解决真实场景中需求表达不规范、Mapping 格式不统一、业务口径需要经验注入的问题。Skill 选择器和生成前表结构分析进一步增强了系统对业务场景和元数据的理解能力。



图 10：智能体增强生成样例加载后的整体界面，展示需求、Skill、表结构辅助与 Mapping 的组合输入

3.5 智能体增强生成的核心优势

智能体增强生成的核心优势不在于简单调用大模型，而在于通过规则草稿、Skill、Memory 和表结构分析对模型进行逐步约束。系统首先由本地规则引擎解析 Mapping 并生成可解释的 SQL 草稿，再把用户需求、Mapping 原文、规则草稿、业务 Skill、历史经验 Memory 和表结构分析结果组合为受控 Prompt。模型不是从零开始自由生成 SQL，而是在已经确定的字段来源、目标字段、统计口径和表结构上下文中完成补全、修正和优化。

与纯大模型直接生成 SQL 相比，本系统更强调生成前的业务约束。纯大模型通常只能根据用户描述推断 SQL，当需求描述不完整或业务口径没有写清楚时，容易出现字段选择不准确、分区条件遗漏、指标口径偏移或输出格式不可解析等问题。当前版本将默认 Skill 调整为更贴合托管清核算业务的产品清算汇总、托管清算对账、托管估值核算、资

金划拨核验、账户余额对账、份额登记确认、清算失败原因分析和清算 KPI 统计，相当于告诉模型当前任务属于哪类业务场景、应该优先遵循哪类核验口径和生成策略。

表结构分析进一步强化了生成前的元数据约束。真实数据研发中，仅凭字段名很难判断字段角色，开发人员需要了解哪些字段适合 Join、哪些字段代表时间、哪些字段是分区字段、哪些字段属于指标或维度。系统在生成前分析表结构后，会把表用途、主键候选、Join Key、时间字段、分区字段、指标字段和维度字段注入模型上下文，使模型在生成 SQL 时能够参考真实表结构含义，而不是仅凭字段名称进行猜测。这样可以降低 Join 粒度不一致、分区过滤缺失和指标汇总错误的风险。

增强模式还体现了对上下文污染的控制。系统并不是把所有材料无限制地交给模型，而是将 Mapping、Skill、Memory 和表结构分析按模块组织，只注入与当前生成任务相关的内容。规则草稿提供 SQL 骨架，Skill 提供业务策略，表结构分析提供字段角色，Memory 提供可复用经验，每类上下文都有明确用途。通过这种上下文分层，模型既能获得足够业务信息，又不会被无关材料干扰。

因此，本系统的智能体增强不是“模型替代规则”，而是“规则约束模型”。规则引擎负责稳定性，模型负责语义理解，Skill 负责业务经验，表结构分析负责元数据理解，校验器负责生成后复核。这种组合更贴近真实企业数据开发过程，也更适合向评委说明系统为什么不是一个简单的 SQL 生成页面，而是一个带上下文控制和质量闭环的研发辅助工作台。

对比维度	纯大模型直接生成 SQL	本系统智能体增强生成
生成依据	主要依赖用户输入的自然语言描述，缺少稳定结构化底座。	先由规则引擎生成 SQL 草稿，再由智能体模型基于草稿和上下文增强。
业务口径	需要模型自行猜测业务模式，容易遗漏企业内部统计习惯。	通过 Skill 显式注入产品汇总、同比、环比、用户分层、KPI 等业务场景。
字段理解	通常只根据字段名推断含义，缺少表用途和字段角色信息。	通过表结构分析识别 Join Key、时间字段、分区字段、指标字段和维度字段。
输入适应性	非标准 Mapping 可能导致理解偏差，生成结果不稳定。	支持 Excel、CSV、Markdown、JSON、TXT 等材料，并可辅助修复 Mapping 结构。
工程稳定性	外部模型失败时容易中断流程。	模型失败时仍可回退到本地规则生成和启发式分析。

3.6 规则约束与质量校验设计

SQL 生成模块不仅提供生成能力，也通过规则约束和质量校验控制输出结果。该设计的出发点是：真实数仓研发不能只追求生成速度，还必须保证 SQL 结构清晰、字段来源可追溯、输出字段不遗漏、聚合口径不漂移，并且在模型服务不可用时仍然能够产生可审阅的规则草稿。因此，系统把约束分布在生成链路的三个阶段，分别是生成前的输入与上下文约束、生成中的规则草稿约束，以及生成后的 SQL 规范校验。

生成前的约束主要解决“模型应该看什么”的问题。系统会先解析 Mapping，检查目标表、来源表、目标字段、关联关系、过滤条件和分区字段等基础信息是否存在；如果启用 Skill 和表结构分析，系统还会把业务策略和元数据角色整理为结构化上下文。这样做可以避免模型在信息不足时自行补全不存在的业务假设，也可以避免把无关表结构或无关经验混入当前任务，形成上下文污染。

生成中的约束主要解决“模型应该怎么写”的问题。规则引擎生成的 SQL 草稿为模型提供稳定骨架，提示词要求模型基于草稿增强，而不是完全重写。输出要求会限制 SQL 关键字、字段别名、WITH 组织方式、目标字段覆盖、GROUP BY 和返回格式，使模型输出尽量保持可解析、可审阅、可继续校验。

生成后的约束主要解决“模型写完后如何把关”的问题。当前规则配置来源于 config/sql_rules.json，校验逻辑由 src/sql_style.py 执行，检查内容包括必需结构块、主查询 SELECT *、字段覆盖、来源别名、聚合 GROUP BY 等。校验逻辑不依赖模型，因此无论使用模板生成还是智能体增强生成，最终结果都会进入同一套质量检查流程。这样可以避免智能体增强只停留在语义补充层面，而缺少工程系统应有的确定性约束。

约束类型	当前规则	作用
Mapping 结构约束	必须包含 task_name、target_table、target_partition、sources、target_columns；sources 和 target_columns 不能为空。	保证系统生成前先拿到目标表、来源表、分区和字段映射等关键结构，避免基于不完整材料直接生成 SQL。
SQL 骨架约束	规则引擎按 WITH、INSERT OVERWRITE TABLE、PARTITION、SELECT、FROM、JOIN、WHERE、GROUP BY 组织 SQL。	让 SQL 初稿具备稳定结构，便于开发人员和评审人员快速定位来源表、过滤条件、聚合逻辑和写入目标。
平台规范约束	SQL 必须包含必要结构块，目标表名和字段别名保持小写，SQL 以分号结尾，主查询禁止 SELECT *。	使生成 SQL 更符合数仓平台常见规范，减少格式不统一和低质量查询写法。
聚合与字段约束	存在 SUM、COUNT、MAX、MIN、AVG 等聚合表达式时必须包含 GROUP BY；生成 SQL 必须覆盖 Mapping 中全部 target_columns。	防止指标聚合口径缺失或目标字段漏出，保证生成结果与 Mapping 输出口径一致。
来源别名约束	表达式、Join 条件和过滤条件中引用的来源别名必须在 sources 中定义。	减少字段来源写错、别名拼写错误和 Join 条件引用非法来源的问题。
智能体增强约束	智能体增强先使用规则引擎生成草稿，再把规则草稿、Mapping、需求、Skill、Memory 和表结构分析一起交给模型。	避免纯大模型从零生成导致口径漂移，使模型增强建立在可追溯的规则底座和业务上下文之上。

3.7 智能体如何驾驭模型输出

智能体驾驭模型输出的关键，是把模型放进一条受控链路，而不是让模型直接决定最终结果。系统首先通过输入组织控制模型能看到的内容，再通过提示词控制模型应遵循的格式和业务规则，最后通过规则校验控制模型输出能否被接受。这个过程本质上是一种上下文控制：控制模型依据哪些材料推理，限制模型在哪些边界内生成，并用本地规则把输出重新拉回工程标准。

在输入层面，系统会明确提供用户真正要统计的指标、Mapping 中的目标字段、来源表和 Join 条件、当前选择的 Skill、团队已有的通用 Memory，以及表结构中识别出的分区字段、指标字段和维度字段。模型获得的是经过筛选和组织的任务上下文，而不是杂乱的原始资料。这样可以减少模型自行猜测，也能降低因为上下文过多、上下文无关或上下文冲突导致的输出漂移。

在提示词层面，系统会告诉模型必须基于规则草稿进行增强，必须覆盖 Mapping 的目标字段，必须遵守 SQL 结构和命名规范，并且只返回可解析的 SQL 或 JSON。这些约束让模型输出更容易被后端继续处理，也让评审人员能够理解模型增强到底发生在哪些范围内，而不是把生成结果看成不可解释的黑箱。

在输出层面，系统通过规则校验和字段检查对生成结果进行复核。如果结果存在字段遗漏、结构缺失、别名异常或聚合规则问题，系统会在校验信息中暴露风险，后续可以依据规则提示进行修正。由此形成的链路是：先限定输入，再约束生成，最后校验输出。这样的设计可以有效避免大模型自顾自输出，使智能体增强更像真实研发流程中的辅助审查员，而不是一个单纯的聊天式 SQL 生成器。

设计问题	系统做法	解决效果
模型不知道业务背景	注入用户需求、Skill 和 Memory。	让模型按产品汇总、KPI、用户分层等明确业务场景生成。
模型不知道字段含义	注入表结构分析结果，包括表用途、Join Key、分区字段、指标字段和维度字段。	降低字段选错、Join 粒度不一致和分区遗漏风险。
模型容易自由发挥	先给规则引擎生成的 SQL 草稿，再要求基于草稿增强。	限制生成边界，使结果保留可追溯 SQL 骨架。
模型输出格式不稳定	提示词要求 SQL 场景只返回 SQL，分析场景返回合法 JSON。	便于后端解析、展示和继续校验。
模型可能漏字段	生成后执行字段覆盖检查，要求覆盖 Mapping 中全部 target_columns。	保证输出字段与 Mapping 目标一致。
模型服务可能失败	回退规则引擎或本地启发式分析。	保证系统演示和基础功能不中断。

4. 版本对比模块

4.1 模块功能说明

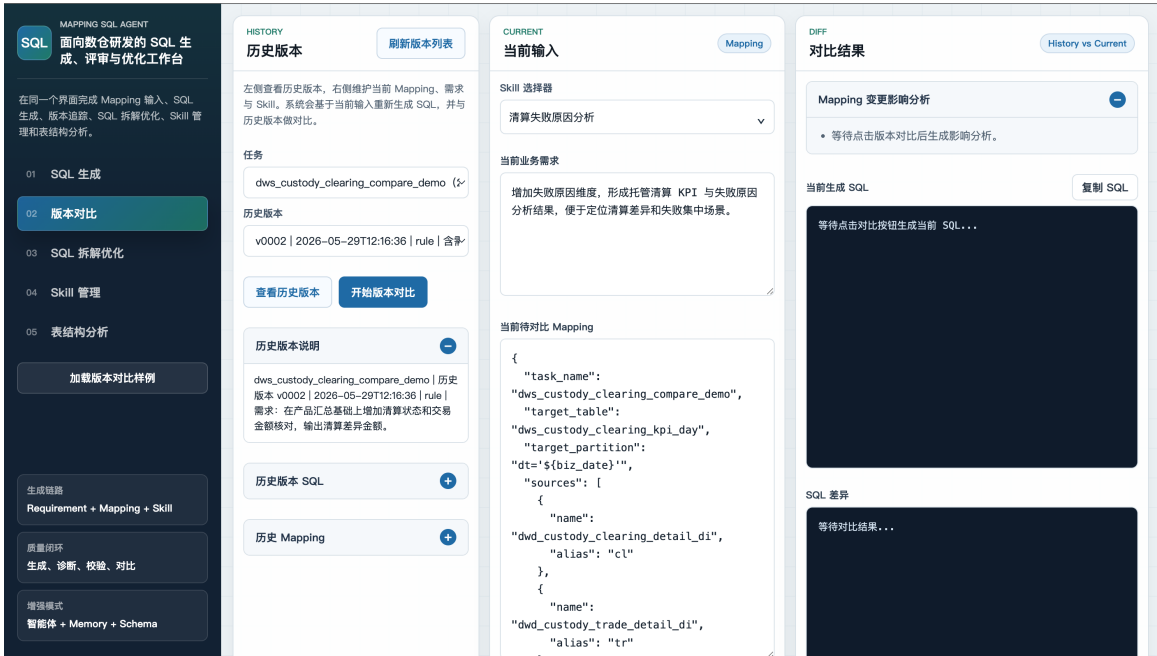


图 11：版本对比工作区整体页面

版本对比模块用于解决需求迭代后的影响追踪问题。真实数据开发中，需求经常从单一维度汇总演进为多维度、多过滤条件、多指标输出。如果只查看最终 SQL，评审人员很难快速判断变化是否合理。该模块通过历史版本存储、当前版本生成、SQL 文本差异和 Mapping 结构影响分析，帮助评审人员理解变更范围。

4.2 样例加载内容

版本对比样例使用任务 `dws_custody_clearing_compare_demo`。历史版本 `v1` 按产品汇总每日托管清算金额和清算笔数；历史版本 `v2` 增加清算状态、交易金额核对和差异金额输出；当前版本 `v3` 继续新增产品维表、失败原因维度、失败笔数和清算 KPI 统计口径。该样例模拟真实清算业务中常见的新增核验维度、调整对账口径、补充异常原因和完善指标字段的变化过程。

4.3 关键组件说明

版本对比模块的关键组件包括任务选择、历史版本选择、历史版本说明、当前 Mapping 输入、当前需求与 Skill 配置、当前生成 SQL、SQL 差异和 Mapping 影响分析。系统不是只比较两段 SQL 文本，而是同时展示 Mapping 结构变化和 SQL 文本变化，使评审人员能够判断新增字段、来源表、过滤条件和指标口径是否合理。



图 12：当前版本输入区

图 13：历史版本选择区



图 14：对比结果区

4.4 模块亮点

该模块的价值不只是文本 diff，而是把 SQL 差异和 Mapping 结构变化结合起来。系统能够说明新增了哪些来源表、Join、过滤条件、目标字段和指标口径，适合用于需求变更评审、SQL 改动复核和历史口径追踪。

5. SQL 拆解优化模块

5.1 模块功能说明



图 15: SQL 拆解优化工作区整体页面

SQL 拆解优化模块面向已有 SQL 的审查场景。真实项目中并非所有 SQL 都从零生成，开发人员经常需要接手历史 SQL、检查他人 SQL 或对复杂 SQL 做优化。该模块通过结构拆解、作用分析、优化建议和优化后 SQL 展示，帮助评审人员快速理解 SQL 逻辑。

5.2 样例加载内容

SQL 分析样例会加载一段托管清算对账 SQL，自动选择“托管清算对账”Skill，并启用表结构分析辅助。表结构样例使用托管清算明细表 DDL，用于展示 SQL 文本、业务 Skill 和表结构上下文如何共同参与 SQL Review。通过这个样例，评审者可以看到系统如何识别清算 SQL 的业务用途、聚合维度、金额口径、状态字段、差异金额和失败原因，并给出更贴近真实对账场景的优化建议。

5.3 关键组件说明

SQL 拆解优化模块的关键组件包括 SQL 文件上传、SQL 输入区、Skill 选择器、生成前表结构分析、SQL 作用分析、结构拆解、优化建议、优化后 SQL 和优化前后差异。该组合使评审人员可以同时看到 SQL 的业务用途、结构组成、潜在问题和修改方向。

	上传 SQL 文件 支持 '.sql' 或 '.txt'，系统会自动读取文件内容并填充到编辑区。 选择 SQL 文件 Skill 选择器 托管清算对账 生成前分析表结构 上传表结构文件 支持 '.xlsx'、'.csv'、'.json'、'.sql'、'.txt'。 选择表结构文件 分析后辅助优化 CREATE TABLE dwd_custody_clearing_detail_di (clear_id STRING COMMENT '清算流水号', trade_id STRING COMMENT '交易流水号', product_id STRING COMMENT '产品编号', account_id STRING COMMENT '托管账户号', trade_dt STRING COMMENT '交易日期' 分析并优化 SQL WITH clearing AS (SELECT		SQL 输入与分析配置区 该区域用于上传或粘贴 SQL，并配置 Skill 和表结构辅助，支持从单纯 SQL 文本扩展到业务上下文分析。
	SQL 作用分析 等待分析结果... SQL 结构拆解 等待分析结果... 增强上下文 Skill: 托管清算对账 说明: 适用于交易流水、清算流水、到账结果之间的金额、笔数、状态和差异核验。 Memory: 已随 Skill 自动注入 记忆条目: 待分析后展示具体条目 表结构增强 已启用生成前表结构分析，待分析后展示表用途、推荐 Skill 和可复用场景。		作用分析与优化建议区 该区域展示 SQL 作用分析和优化建议，帮助评审人员快速理解 SQL 的业务目标和潜在优化方向。
	优化建议 将LEFT JOIN改为INNER JOIN，确保清算记录与交易记录匹配，避免因缺失交易金额导致差异计算为NULL。 使用COALESCE(t.trade_amt, 0)包裹SUM，防止LEFT JOIN未匹配时trade_amt为NULL导致diff_amt异常。 在SELECT中增加account_id、trade_dt、clear_dt、fail_reason字段，提升对账粒度和问题定位能力，符合业务规则保留日期和失败原因。 将GROUP BY扩展为c.product_id, c.account_id, c.trade_dt, c.clear_dt, c.clear_status, c.fail_reason，避免合并不同原因或日期的记录。 移除CTE直接引用表，简化SQL结构并减少一次数据扫描，已在优化SQL中体现。 目标表dws_custody_clearing_reconcile_day若已有不同粒度，需调整表结构和分区策略以适应新增维度。 优化后 SQL 复制 SQL INSERT OVERWRITE TABLE dws_custody_clearing_reconcile_day PARTITION (dt = '\${biz_date}') SELECT c.product_id,		优化后 SQL 区 该区域展示优化后的 SQL 文本，便于开发人员复制、复核或继续调整。

图 18：优化后 SQL 区

5.4 模块亮点

该模块的亮点在于把 SQL Review 从人工阅读扩展为结构化分析。系统能够识别 SELECT、FROM、JOIN、WHERE、GROUP BY 等片段，并结合 Skill 和表结构判断字段角色、过滤条件和聚合口径，使优化建议更贴近业务和性能要求。

6. Skill 管理模块

6.1 模块功能说明

Skill 管理模块用于维护系统中的业务 Skill，使业务经验可以被查看、编辑、生成和复用。该模块新增后，系统不再只有固定的几个通用 Skill，而是可以根据托管、清算、核算等业务场景持续沉淀新的 Skill。页面左侧展示已有 Skill，用户点击后可以在中间编辑器查看和修改 Skill ID、名称、适用场景、SQL 生成偏好、示例场景和业务规则；右侧 Skill 生成器则用于根据业务场景、表结构或 Mapping 材料生成新的 Skill 草稿。



图 19: Skill 管理工作区整体页面，展示已有 Skill、Skill 编辑器和 Skill 生成器

6.2 样例加载内容

Skill 管理样例使用托管清算对账业务。点击加载样例后，系统会填入一段托管清算场景说明和表结构材料，内容包括产品、托管账户、交易日期、清算日期、清算状态、交易金额、清算金额、差异金额、失败原因和分区日期等字段。点击“生成 Skill 草稿”后，

系统会把这些业务要素整理为结构化 Skill，包括 Skill 名称、适用说明、SQL 生成偏好、示例场景和业务规则。

EDITOR

Skill 编辑器

草稿

Skill ID

custody_clearing_reconcile_6940

Skill 名称

托管清算对账

适用场景说明

适用于托管清算对账场景下的 SQL 生成、拆解和优化，强调业务口径、字段角色、分区过滤和结果可复核。典型需求：要求优先使用 dt 或 clear_dt 过滤，金额字段使用 SUM，状态

SQL 生成偏好 / 模式

按产品、账户、交易日期、清算日期、状态分组：输出金额、笔数、成功笔数、失败笔数、差异金额等指标；优先使用分区日期、清算日期、交易日期、状态过滤

示例场景（每行一条）

按产品、账户、交易日期、清算日期、状态统计金额、笔数、成功笔数、失败笔数、差异金额
对托管清算对账结果进行字段覆盖、分区过滤和异常状态复核

业务规则（每行一条）

生成 SQL 时应优先保留业务日期或分区日期条件，避免全表扫描。
金额、份额、净值等指标字段应明确聚合方式，并注意空值处理。

保存 Skill

删除 Skill

FACTORY

Skill 生成器

托管清算核算

业务场景

托管清算场景下，按产品、交易日期、清算日期和清算状态统计清算金额、成功笔数、失败笔数和差异金额，并识别清算失败原因。

可选：表结构 / Mapping / 样例 SQL

CREATE TABLE
dwd_custody_clearing_detail_di (
product_id STRING COMMENT '产品编号',
account_id STRING COMMENT '托管账户'

可选：生成要求

要求优先使用 dt 或 clear_dt 过滤，金额字段使用 SUM，状态字段需要区分成功、失败和处理中，输出结果要便于对账复核。

生成 Skill 草稿

加载托管清算样例

草稿预览

{
"id": "custody_clearing_reconcile_6940",
"name": "托管清算对账",
"description": "适用于托管清算对账场景下的 SQL 生成、拆解和优化，强调业务口径、字段角色、分区过滤和结果可复核。典型需求：要求优先使用 dt 或 clear_dt 过滤，金额字段使用 SUM，状态字段需要区分成功、失败和处理中，输出结果要便于对账复核。",

图 20: Skill 生成器草稿生成结果，展示托管清算对账场景如何被整理为结构化 Skill

6.3 关键组件说明

已有 Skill 列表用于快速查看当前系统可用的业务模式，列表展示中文名称和英文 ID，详细内容由右侧编辑区承接。Skill 编辑器用于维护结构化字段，保存后会写回本地业务记忆配置，并同步刷新 SQL 生成、版本对比和 SQL 拆解优化中的 Skill 选择器。Skill 生成器用于降低创建门槛，用户不需要一开始就知道 Skill 应该如何组织，只需要提供业务场景和相关材料，系统就能生成可继续编辑的草稿。

SKILLS

已有 Skill

新建 Skill

Skill 用于把托管、清算、核算等业务经验沉淀为可复用上下文，SQL 生成和 SQL 拆解优化会将选中的 Skill 注入智能体提示词。

无
none

同比汇总
yoy_summary

环比汇总
mom_summary

产品维度汇总
product_aggregation

用户分层
user_segmentation

KPI统计
kpi_metrics

托管清算对账
custody_clearing_reconcile

托管估值核算
custody_valuation_accounting

资金划拨核验
fund_transfer_verify

EDITOR

Skill 编辑器

编辑

Skill ID

product_aggregation

Skill 名称

产品维度汇总

适用场景说明

适用于按产品、产品层级或产品组合做销售、客户、收益等指标汇总。

SQL 生成偏好 / 模式

GROUP BY product_id, product_name, product_category

示例场景（每行一条）

按产品、产品分类聚合销售额和订单数
适合产品经营看板

业务规则（每行一条）

清算类 SQL 应优先保留清算日期或分区日期过滤

图 21: Skill 编辑器，展示 Skill 名称、英文 ID、适用场景、生成偏好和业务规则

6.4 模块亮点

该模块的亮点在于把 Skill 从固定选项升级为可维护的业务知识资产。对于评委而言，这一点能够说明系统不是简单把需求和 Mapping 喂给大模型，而是允许用户把托管清算、估值核算、资金划拨等业务经验沉淀为结构化规则，再由 PromptBuilder 注入智能体上下文。新增或修改后的 Skill 会影响后续 SQL 生成和 SQL 拆解优化，形成“业务经验输入—Skill 结构化—智能体上下文约束—生成结果复核”的闭环。

7. 表结构分析模块

7.1 模块功能说明

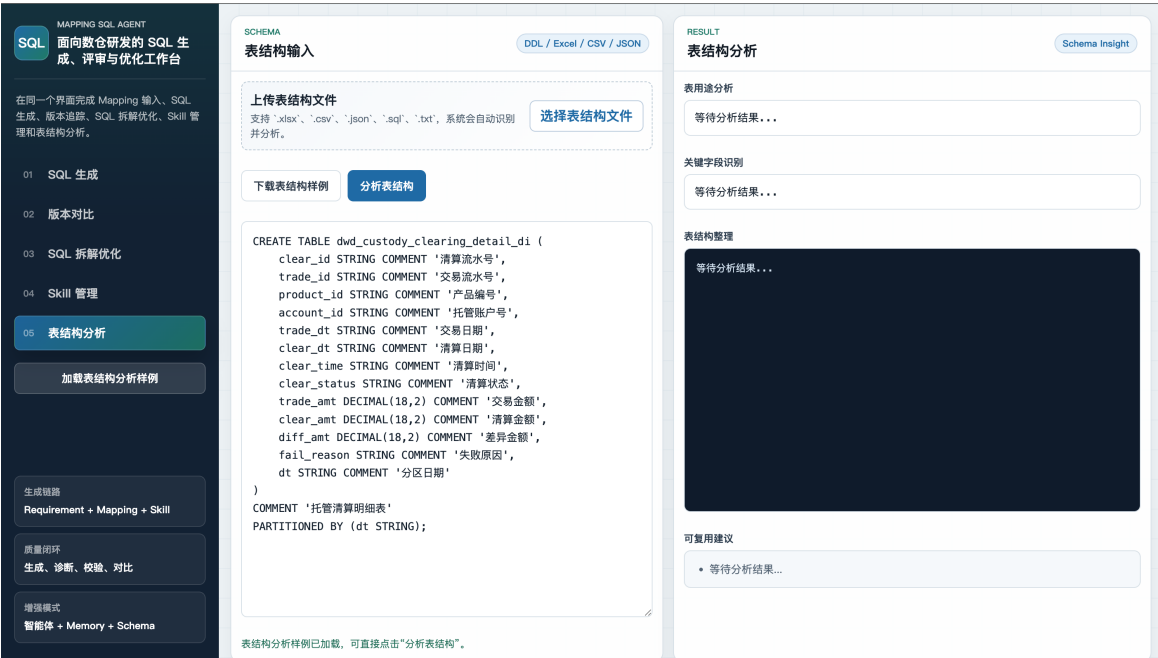


图 22：表结构分析工作区整体页面

表结构分析模块用于把 DDL、Excel、CSV、JSON 或文本形式的表结构转换为结构化元数据理解结果。系统不会执行 CREATE TABLE，也不会创建数据库表，而是把 DDL 当作表结构文档解析。该设计符合真实企业中先读取元数据、再辅助 SQL 开发和评审的流程。

7.2 样例加载内容

表结构样例使用 dwd_custody_clearing_detail_di 的 CREATE TABLE DDL，字段包括 clear_id、trade_id、product_id、account_id、trade_dt、clear_dt、clear_status、trade_amt、clear_amt、diff_amt、fail_reason 和 dt。系统会将其识别为托管清算明细事实表，并提取清算金额、交易金额、差异金额等指标字段，产品和账户等维度字段，清算状态和失败原因等复核字段，以及 dt 分区日期字段。

7.3 关键组件说明

表结构分析模块的关键组件包括表结构文件上传、DDL 或表结构文本输入、表用途分析、关键字段识别、表结构整理和可复用建议。系统会从字段名、字段类型和字段注释中识别主键候选、Join Key、时间字段、分区字段、指标字段和维度字段，为 SQL 生成和 SQL Review 提供元数据依据。

上传表结构文件 支持 '.xlsx'、'.csv'、'.json'、'.sql'、'.txt'，系统会自动识别并分析。 选择表结构文件 下载表结构样例 分析表结构 <pre>CREATE TABLE dwd_account_trade_detail_di (trade_id STRING COMMENT '交易流水号', user_id STRING COMMENT '客户号', account_id STRING COMMENT '账户号', product_id STRING COMMENT '产品编号', trade_dt STRING COMMENT '交易日期', trade_time STRING COMMENT '交易时间', trade_amt DECIMAL(18,2) COMMENT '交易金额', channel_code STRING COMMENT '渠道编码', city_name STRING COMMENT '城市名称', dt STRING COMMENT '分区日期') COMMENT '账户交易明细表' PARTITIONED BY (dt STRING);</pre> 图 23：表结构输入区	表结构输入区 该区域支持粘贴 DDL 或上传表结构文件，是系统理解字段和表用途的输入入口。
表用途分析 存储每日账户交易明细的详情表，用于支撑交易相关的分析统计与下游应用。 关键字段识别 主键候选: trade_id Join Key: user_id、account_id、product_id 时间字段: trade_dt、trade_time 分区字段: dt 指标字段: trade_amt 维度字段: channel_code、city_name	表用途与关键字段区 该区域优先展示表用途分析和关键字段识别，帮助评审人员先理解这张表服务于什么业务场景。
表结构整理 <pre>trade_id STRING 交易流水号 user_id STRING 客户号 account_id STRING 账户号 product_id STRING 产品编号 trade_dt STRING 交易日期 trade_time STRING 交易时间 trade_amt DECIMAL(18,2) 交易金额 channel_code STRING 渠道编码 city_name STRING 城市名称 dt STRING 分区日期</pre> 图 25：结构整理与复用建议区	结构整理与复用建议区 该区域展示字段清单、字段类型、字段注释和可复用建议，为后续 SQL 生成和 SQL Review 提供元数据依据。

7.4 模块亮点

表结构分析使系统具备元数据理解能力。它能够识别主键候选、Join Key、时间字段、分区字段、指标字段和维度字段，并将这些结果反向服务于 SQL 生成和 SQL 拆解优化。该能力使系统不只是处理 SQL 字符串，而是能够理解字段在业务中的角色。

8. 系统完整性与实际业务贴合度

从功能完整性看，系统覆盖了数据研发中从需求和 Mapping 到 SQL 初稿，再到版本追踪、SQL Review、业务经验沉淀和表结构理解的完整链路。SQL 生成解决从需求到代码的初稿生产问题；版本对比解决需求变更后的影响追踪问题；SQL 拆解优化解决已有 SQL 的阅读和评审问题；Skill 管理解决业务经验维护和复用问题；表结构分析解决字段含义和元数据理解问题。五个模块之间能够形成闭环，而不是彼此孤立。

从业务贴合度看，系统输入材料覆盖真实研发中常见的 JSON Mapping、Excel Mapping、CSV、Markdown、TXT、SQL 文件和 DDL 文本。系统内置样例围绕事实表、维表、Join、过滤条件、聚合指标、分区字段、业务维度、字段注释和托管清核算业务规则展开，能够对应托管清算对账、产品清算汇总、清算失败原因分析、估值核算、资金划拨、余额对账、需求变更评审和 SQL 优化等真实场景。由于 Skill、Memory 和样例数据都是可配置、可维护的结构，后续也可以用同样方法扩展到其他业务部门，例如风险监控、客户经营、交易运营、财务核算和管理驾驶舱指标加工。

从工程设计看，系统采用规则引擎和智能体增强结合的方式。规则模式保证稳定性和可解释性，增强模式负责处理自然语言需求、非标准输入、Skill 注入和表结构上下文。与纯大模型直接生成 SQL 不同，本系统通过 Skill 明确业务场景，通过 Skill 管理沉淀托管清算和核算经验，通过表结构分析明确字段角色，通过规则草稿约束 SQL 骨架，使模型增强能力建立在可追溯的工程上下文之上。模型失败时系统仍可回退到本地规则或本地启发式分析，体现了原型系统对可用性和兜底机制的考虑。

9. 项目关注要点

评审阅读时可以重点关注三个方面。第一，系统覆盖的是完整数据研发链路，而不是单一 SQL 生成页面；第二，AI 能力被规则草稿、Skill、Memory、表结构分析和规则校验逐步约束，避免模型自由发挥；第三，Skill 管理模块使托管清算、估值核算、资金划拨等业务经验能够持续沉淀，并反向服务后续生成和优化。

关注点	材料中对应体现
是否有完整业务流程	第二章和第七章说明系统覆盖生成、对比、拆解优化和表结构分析。
是否贴近真实数据研发	样例围绕产品销售、版本变更、SQL Review 和账户交易明细表结构展开。
是否有智能增强能力	SQL 生成模块展示 智能体增强、Skill、Memory 和 Schema Insight 的组合，并说明其相较纯大模型生成在业务口径、字段理解和工程稳定性上的优势。
是否有工程稳定性	规则模式、本地兜底和模型失败回退机制保证系统不完全依赖外部模型。
页面是否可理解	各模块均提供整体截图、关键组件截图和组件作用说明。